ResDish
FOOD RESCUE

Going Live Checklist

Moving ResDish from test mode to real payments, real email and a public website

Prepared October 2026 · for the ResDish owner

Tick each box as you go. Never write passwords or secret keys in this document.

Before you start

ResDish needs **no code changes** to go live. Going live means connecting real outside services through the settings file (`.env.local`, or your host's environment variables), plus a few business and legal steps. Work through the sections in order; each one ends with a short check.

Three kinds of "test"

Mode	How you get it	What happens
ResDish test payments	No Stripe keys in `.env.local` (what you run today).	Everything is simulated inside the app. Restaurants "connect" a practice account with a MOCK BANK; there is no Stripe dashboard.
Stripe test mode	Stripe keys that start with `sk_test_` and `pk_test_`.	The real Stripe system with pretend money. Onboarding, dashboards, transfers and refunds all work. Use test card 4242 4242 4242 4242.
Live	Stripe keys that start with `sk_live_` and `pk_live_`.	Real cards are charged and real money is paid out to restaurants' banks.

Recommended path: **test payments** → **Stripe test mode** → **live**. Do a full dry run in Stripe test mode before you switch to live keys.

Your order of work

#	Section	Time needed (roughly)
1	Business and legal (Washington)	Days to weeks (start first)
2	Domain and email	1 to 2 hours, plus DNS wait
3	Supabase (database and logins)	1 hour
4	Hosting (public website)	1 to 2 hours
5	Stripe in test mode, then live	2 hours, plus Stripe verification
6	Maps	20 minutes
7	Security	30 minutes
8	Launch-day test and go	1 hour

1. Business and legal (Washington)

Start these first: they take the longest and Stripe live mode needs your business details.

- ☐ Register the business (LLC or corporation) and get an **EIN** from the IRS.
- ☐ Get a **Washington business license** (Business Licensing Service, through the WA Department of Revenue, My DOR).
- ☐ Register with the **WA Department of Revenue** for sales tax. As a marketplace facilitator, ResDish collects and remits sales tax on food orders and on restaurant plan fees.
  - Each tax return: owner console → **Sales tax** tab → choose the dates → **CSV** or **PDF**. It is split by city, ZIP and rate.
- ☐ Open a **business bank account** (Stripe pays ResDish's share into it).
- ☐ Have a **Washington-licensed attorney** review the Customer Terms, Restaurant Partner Agreement and Privacy Policy (in the app at `/legal/...`, text in `src/lib/legal/documents.ts`). If the text changes, the version number must change too, and everyone accepts the new version at their next login.
- ☐ Set your real company details in `.env.local`: `LEGAL_ENTITY_NAME` (registered company name), `SUPPORT_EMAIL`, `LEGAL_ADDRESS`. They appear in the legal documents, invoices and emails.
- ☐ Consider **business insurance** (general liability; ask about marketplace and food-related cover).
- ☐ Restaurants must hold their food permits (health department). The Partner Agreement already requires it; you may want to ask for a permit number when you approve them.

2. Domain and email

- ☐ Buy your domain (for example `bite-wise.org`) if you don't own it yet.
- ☐ Pick an email service that can send as your domain. **Gmail cannot send as no-reply@bite-wise.org.** Good options: **Resend**, **Postmark**, **Amazon SES**, or **Google Workspace** (mailboxes plus sending).
- ☐ Add the DNS records the email service gives you: **SPF**, **DKIM** and **DMARC**. Without them, emails land in spam.
- ☐ Put the SMTP details in `.env.local`:
  - `SMTP_HOST`, `SMTP_PORT` (usually 587), `SMTP_USER`, `SMTP_PASSWORD`
  - `EMAIL_FROM`, for example `ResDish <hello@bite-wise.org>`
- ☐ Create a **support mailbox** (for example `support@bite-wise.org`) and set `SUPPORT_EMAIL` to it.

> **Check**
>
> Sign up a test customer with your own email: the confirmation email should arrive from your domain, with the ResDish logo, not in spam.

3. Supabase (database and logins)

- ☐ Use a dedicated **production** project on supabase.com. The **Pro plan** gives daily backups and doesn't pause when quiet.
- ☐ Copy its keys into `.env.local`: `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`, `SUPABASE_SECRET_KEY` (Project Settings → API).
- ☐ Create the tables: `npx supabase link` (choose the production project), then `npx supabase db push`.
- ☐ **Do not run** `npm run seed` on production: it adds the demo restaurants and demo accounts.
- ☐ Create your owner account: `npm run create-admin`.
- ☐ Authentication → Sign In / Providers → Email: turn **Confirm email** on.
- ☐ Authentication → URL Configuration: set **Site URL** to `https://your-domain`.
- ☐ Authentication → Emails → SMTP Settings: enter the same email service as in section 2.
- ☐ Install the branded backup sign-up email: add `SUPABASE_ACCESS_TOKEN` to `.env.local` and run `npm run email:template`.

> **Check**
>
> The owner console opens with your admin login, and its tabs are empty (no demo data).

4. Hosting (public website)

Customers, restaurants and Stripe need to reach the site over **HTTPS** on your domain. A computer at home or in the office (with a private network address) is not reachable from the internet.

Option A: Vercel (easiest)

- ☐ Create a Vercel project from the GitHub repository (`umershahid84/RescueBites`, branch `main`).
- ☐ Add every setting from `.env.example` in Vercel → Settings → Environment Variables, plus a long random `CRON_SECRET`.
- ☐ Add your domain in Vercel → Domains and follow its DNS instructions (HTTPS is automatic).
- ☐ `vercel.json` already runs the 5-minute scheduled jobs (card holds, missed pickups, plan renewals).

Option B: your own Linux server (VPS)

- ☐ A small Ubuntu server (for example 2 GB RAM) with Node.js 20 or later.
- ☐ Clone the repository, create `.env.local`, run `npm ci`, `npm run build`, then `npm run service:install` (runs ResDish as a service that restarts by itself).
- ☐ Put **Caddy** or **Nginx** in front for HTTPS on your domain.
- ☐ Updates later: `git pull` then `npm run update`.

Both options

- ☐ `NEXT_PUBLIC_SITE_URL=https://your-domain` (confirmation links, kiosk links, Stripe return links and invoice links use it).
- ☐ `TIME_ZONE=America/Los_Angeles`.

> **Check**
>
> Open `https://your-domain` on your phone (not on your home Wi-Fi): the home page and map load, and the padlock shows a valid certificate.

5. Stripe: test mode first, then live

5a. Set up the platform

- ☐ Create the Stripe account for your company and complete **business verification** (legal name, EIN, address, bank account, owner identity).
- ☐ Settings → **Connect**: turn it on, choose **Express** accounts, and complete the **platform profile** (ResDish is a marketplace that pays restaurants for food sold through it).
- ☐ Connect → Settings → Branding: add the ResDish name, logo and colors (restaurants see them during onboarding).

5b. Dry run with test keys

- [ ] Developers → API keys (Test mode): put `sk_test_...` in `STRIPE_SECRET_KEY` and `pk_test_...` in `NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`.
- [ ] Developers → Webhooks → Add endpoint: `https://your-domain/api/stripe/webhook`, with these events:

Event	Listen on
`payment_intent.amount_capturable_updated`	Your account
`payment_intent.payment_failed`	Your account
`account.updated`	Connected accounts

- [ ] Copy the webhook's **signing secret** (`whsec_...`) into `STRIPE_WEBHOOK_SECRET`, then restart or redeploy.
- [ ] As a test restaurant: **Payouts** tab → **Set up payouts with Stripe** → finish Stripe's test onboarding (Stripe shows test values to use). The tab shows "Your Stripe account is ready" and **Open Stripe dashboard** works.
- [ ] As a test customer: order with card `4242 4242 4242 4242` (any future date, any CVC). At the counter, enter the PIN and hand it over.
- [ ] Confirm in Stripe (test mode): the payment, ResDish's application fee, and the transfer to the restaurant.
- [ ] In the owner console: refund that order (Orders tab), and see it in Stripe.
- [ ] A paid plan: subscribe with `4242...`; then try `4000 0000 0000 0002` (declined) to see the delinquent flow.

5c. Switch to live

- [ ] Developers → API keys (Live mode): put `sk_live_...` and `pk_live_...` in `.env.local`.
- [ ] Create the **same webhook again in Live mode** (it has its own signing secret) and update `STRIPE_WEBHOOK_SECRET`.
- [ ] Restart or redeploy. Restaurants connected during test payments or Stripe test mode must click **Set up payouts with Stripe** again to connect for real.

> **Good to know**
>
> Stripe charges its fees on each payment; the Income tab shows ResDish's income before those fees.
>
> Keep `sk_live_...` secret: anyone with it can move money. Only the server should have it.

6. Maps

- ☐ OpenStreetMap's free public tiles are not allowed for a busy site. Sign up with a tile provider: **MapTiler**, **Stadia Maps** or **Mapbox** (all have free tiers).
- ☐ Set `NEXT_PUBLIC_MAP_TILE_URL` and `NEXT_PUBLIC_MAP_ATTRIBUTION` from the provider's instructions. If you choose a dark map style, set `NEXT_PUBLIC_MAP_DARK_FILTER=false`.
- ☐ Restaurant addresses are placed on the map automatically. For restaurants added earlier, run `npm run locate-restaurants`, then `npm run locate-restaurants -- --yes`.

7. Security

- ☐ **Create new secrets** for production: a new Supabase project has its own keys; also create a new Gmail/email app password. (The old Supabase secret key and Gmail app password were shared in a chat, so don't reuse them.)
- ☐ `.env.local` lives only on the server (or in Vercel's settings). Never email it, commit it to GitHub, or paste it into a chat; if you must share it, blank out every value first.
- ☐ Turn on **two-factor authentication** for GitHub, Supabase, Stripe, Vercel (or your server host), your domain registrar and your email service.
- ☐ Use a strong, unique password for your ResDish admin account.
- ☐ Set a long random `CRON_SECRET` (for example from a password manager).

8. Launch day

Do one complete real transaction with a small amount before inviting restaurants.

- [ ] Sign up a **restaurant** with a real email address. Confirm the email, then approve it in the owner console (Restaurants tab). The welcome email arrives with the signed agreement and the kiosk link.
- [ ] Choose a plan as the restaurant (Pioneer Members are free), connect **real** Stripe payouts, and install the kiosk (Android, iPad or Windows PC).
- [ ] Post a small offer (for example $1).
- [ ] Sign up a **customer** with another real email, order it with a real card, pick it up with the PIN at the kiosk. The customer gets the invoice email with the PDF receipt.
- [ ] Refund it from the owner console (Orders tab) to see a refund end to end.
- [ ] Next morning: Stripe shows the payout on its way to the restaurant's bank; the owner console's **Income** and **Sales tax** tabs show the order.
- [ ] Invite your first restaurants: the first 50 to choose a plan are **Pioneer Members** (free).

Settings reference

Everything goes in `.env.local` (Option B) or Vercel → Environment Variables (Option A). The template is `.env.example` in the project.

Setting	Where it comes from	Production value
`NEXT_PUBLIC_SUPABASE_URL`	Supabase → Project Settings → API	https://<project>.supabase.co
`NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY`	Supabase → API keys	sb_publishable_...
`SUPABASE_SECRET_KEY`	Supabase → API keys (secret)	sb_secret_... (keep secret)
`SUPABASE_ACCESS_TOKEN`	supabase.com → Account → Tokens	only for email:template
`SMTP_HOST`, `SMTP_PORT`	Your email service	e.g. smtp.resend.com, 587
`SMTP_USER`, `SMTP_PASSWORD`	Your email service	keep the password secret
`EMAIL_FROM`	You choose	ResDish <hello@your-domain>
`NEXT_PUBLIC_SITE_URL`	Your domain	https://your-domain
`TIME_ZONE`	You choose	America/Los_Angeles
`STRIPE_SECRET_KEY`	Stripe → Developers → API keys	sk_live_... (keep secret)
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY`	Stripe → Developers → API keys	pk_live_...
`STRIPE_WEBHOOK_SECRET`	Stripe → Developers →	whsec_... (live endpoint)

Setting	Where it comes from	Production value
	Webhooks	
`CRON_SECRET`	You create (long random text)	needed on Vercel
`NEXT_PUBLIC_MAP_TILE_URL`	MapTiler / Stadia / Mapbox	from the provider
`NEXT_PUBLIC_MAP_ATTRIBUTION`	Same provider	from the provider
`LEGAL_ENTITY_NAME`	Your registered company	e.g. ResDish LLC
`SUPPORT_EMAIL`	Your support mailbox	support@your-domain
`LEGAL_ADDRESS`	Your business address	City, Washington

Useful commands

Command	What it does
`npx supabase db push`	Creates or updates the database tables (after every update that includes database changes).
`npm run create-admin`	Creates the owner (admin) account.
`npm run email:template`	Installs the branded sign-up email in Supabase (backup copy).
`npm run locate-restaurants`	Finds map positions for restaurant addresses (add `-- --yes` to save).
`npm run update`	On your own server: pulls the latest version, builds it and restarts.
`npm run service:install`	On a Linux server: runs ResDish as a service that restarts by itself.